Digital logic simplified

It won't feel like complicated math

Digital design becomes fun the moment you stop seeing it as "math" and start seeing it as cause-and-effect rules. You're basically teaching a circuit how to make decisions (logic), how to remember (state), and how to behave predictably over time (timing). This chapter builds the mindset you'll need for Verilog and FPGA work later: think in signals, not statements.

Bits, boolean logic, and the only "math" you must know

A bit is a yes/no value: 0 or 1. Digital logic is just "rules for combining bits." The three core boolean operations you'll use constantly:

- **NOT:** flips a bit (0→1, 1→0)
- **AND:** 1 only if both inputs are 1
- **OR:** 1 if either input is 1

Everything more complex – comparators, adders, multiplexers – reduces to these rules.

Try this tonight: build your first truth table (no tools needed)

Goal: Create a rule for "turn on LED if (Button A AND Button B) OR Override is on."

Step-by-step

1. Name your inputs: A, B, O (override). Output: LED.
2. Write all combinations of A, B, O (there are $2^3 = 8$ combos).
3. For each row:
   - Compute A AND B
   - Then compute (A AND B) OR O
4. Mark LED = 1 when that final expression is true.